

AI Customer Support System Complete Project Walkthrough

A comprehensive guide for building a multi-agent customer support chatbot from scratch. For: Two fresh graduates (AI Engineer + Software Engineer) using agentic vibe coding.

Table of Contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [Team Structure & Roles](#)
 4. [Role Guide: AI Engineer](#)
 5. [Role Guide: Software Engineer](#)
 6. [The Runbook Workflow \(How We Work\)](#)
 7. [Tech Stack & Installation Guide](#)
 8. [Open-Source Datasets](#)
 9. [Getting Started \(Day 1\)](#)
 10. [Weekly Milestones](#)
 11. [Runbook File Reference](#)
 12. [Evaluation & Quality Metrics](#)
 13. [Deployment Guide](#)
 14. [Portfolio Presentation Tips](#)
 15. [Glossary](#)
-

1. Project Overview

What We're Building

A **multi-agent AI customer support chatbot** a web application where users can type customer support questions and receive intelligent, context-aware answers from specialized AI agents.

The system uses three specialist agents:

- **Billing Agent** handles payment issues, refunds, subscriptions, invoicing
- **Technical Agent** handles bugs, errors, setup help, feature questions
- **General Agent** handles greetings, feedback, complaints, general inquiries

An **Orchestrator Agent** automatically classifies the user's intent and routes their question to the correct specialist. The specialists answer using a **knowledge base** (RAG Retrieval-Augmented Generation) powered by real customer support data.

Why This Project?

1. **Portfolio impact** multi-agent AI systems are the hottest skill in AI engineering right now. This project demonstrates agent orchestration, RAG, prompt engineering, and full-stack development.

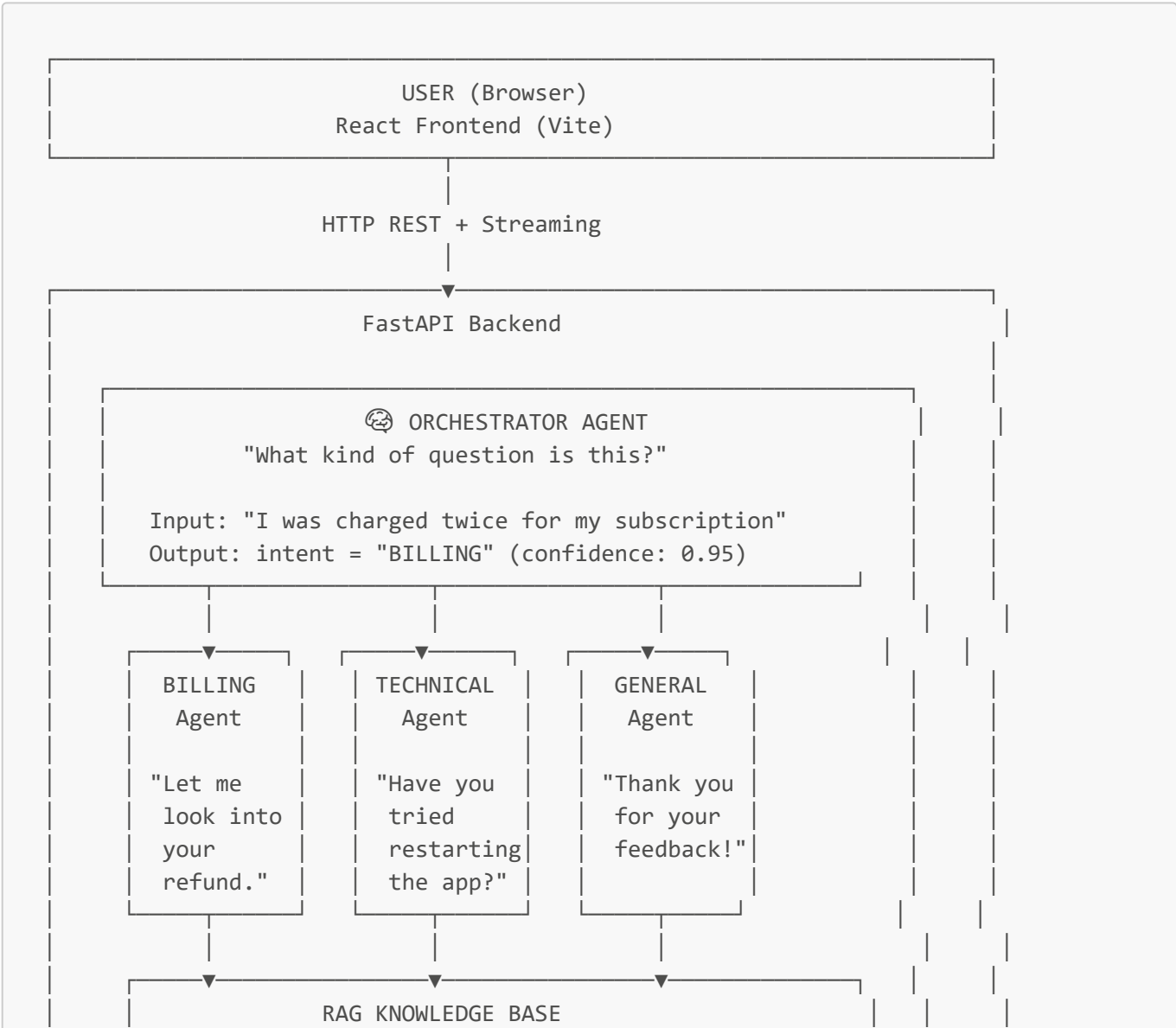
- 2. **Demonstrates end-to-end skills** from data pipeline to deployment, both roles get hands-on experience with the complete software development lifecycle.
- 3. **Real-world relevance** customer support chatbots are deployed by every major tech company. Interviewers understand and appreciate this domain.
- 4. **Demoable** a chat interface is visually engaging and easy to demo in interviews or portfolio reviews.

Constraints

Constraint	How We Handle It
\$0 budget	Free-tier APIs (Gemini, Groq), open-source tools, free hosting (Render, Vercel)
No GPU	RAG instead of fine-tuning; CPU-friendly embedding model; cloud LLM inference
Fresh graduates	Step-by-step specs, beginner-friendly tooling, progressive complexity
2-person team	Clear role separation, independent specs, API contract as the handshake

2. System Architecture

The Big Picture



ChromaDB stores 2,500+ FAQ chunks as vectors.
When a user asks a question, we find the 5 most relevant FAQ answers and feed them to the LLM.

SQLite (app.db)
- conversations
- messages
- feedback

LLM Provider (free tier)
1. Gemini API (primary)
2. Groq API (fallback)
3. Ollama (offline)

How a Message Flows Through the System

```
Step 1: User types "I was charged twice" in React chat UI
Step 2: React sends POST /api/v1/chat → FastAPI backend
Step 3: Backend saves user message to SQLite
Step 4: Backend calls AIService with message + conversation history
Step 5: Orchestrator classifies intent → "BILLING"
Step 6: Billing Agent receives query
Step 7: RAG pipeline embeds query → searches billing_kb in ChromaDB
Step 8: Top 5 relevant FAQ chunks retrieved
Step 9: Prompt assembled: system prompt + FAQ chunks + history + query
Step 10: Prompt sent to Gemini API (or Groq/Ollama fallback)
Step 11: LLM generates response
Step 12: Response returned to FastAPI with sources + agent info
Step 13: Backend saves AI response to SQLite → sends JSON to React
Step 14: React renders response with typing animation + source citations
```

What is RAG? (For Beginners)

RAG = Retrieval-Augmented Generation. Instead of training the AI on our data (which requires expensive GPUs), we:

1. **Store** our FAQ documents as mathematical vectors (embeddings) in a database (ChromaDB).
2. **Search** for the most relevant FAQ when a user asks a question.
3. **Inject** those relevant FAQs into the prompt as context.
4. **Generate** a response that's grounded in our actual knowledge base.

Think of it like giving the AI a cheat sheet before answering a test question. The AI doesn't need to memorize everything it just needs to read the right page at the right time.

```
Without RAG: User asks → LLM guesses (may hallucinate)
With RAG:    User asks → Find relevant docs → LLM reads docs → LLM answers accurately
```

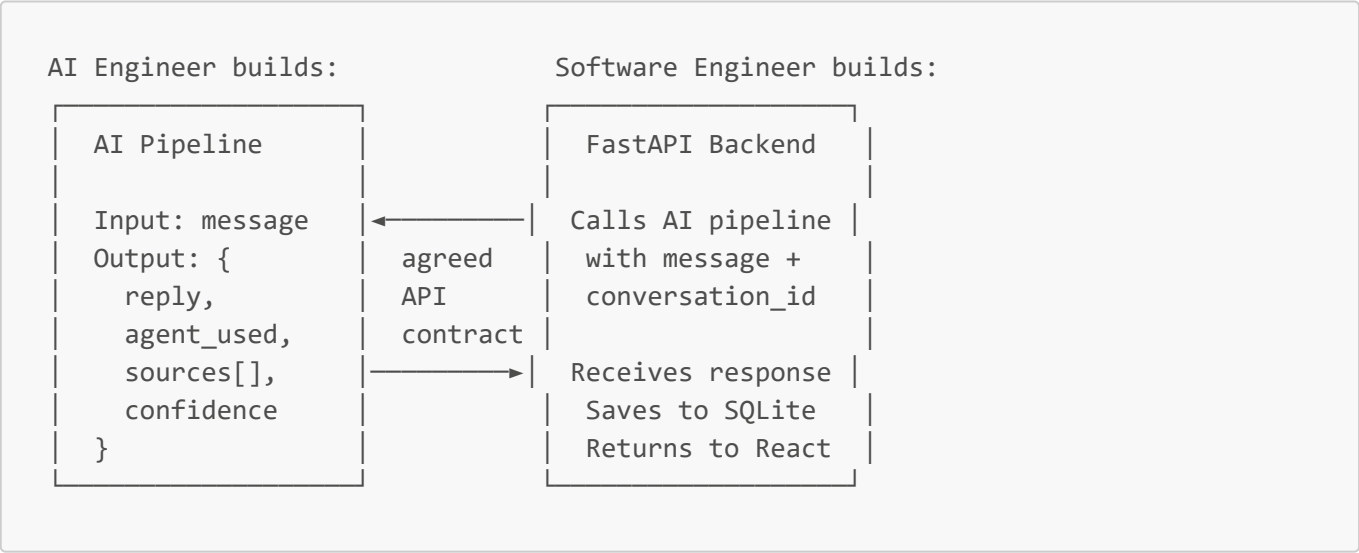
3. Team Structure & Roles

Role Overview

Aspect	AI Engineer (You)	Software Engineer (Friend)
Primary language	Python	TypeScript + Python
Owns	ai/ directory	apps/api/ + apps/web/ directories
Builds	Data pipeline, RAG, agents, prompts, evaluation	Backend API, frontend UI, database, deployment
Writes specs	AI-, EVAL-	API-, WEB-, DEPLOY-*
Shared specs	M-_ (monorepo setup), INT-_ (integration)	M-_ (monorepo setup), INT-_ (integration)
Reviews	All AI-related code + integration specs	All infra/app code + integration specs

The Handshake: API Contract

The most important collaboration artifact is the **API contract** the exact shape of requests and responses between the AI pipeline and the backend. This is defined in `docs/integrations/api-contract.md` and must be agreed upon BEFORE either of you writes code.



Why this matters: With the API contract locked, both of you can work **completely independently** for weeks 2–5. The AI Engineer builds the pipeline that produces the agreed output shape. The Software Engineer builds the backend that consumes it. You only need to sync during integration (week 6).

4. Role Guide: AI Engineer

Your Learning Journey

```

Week 1: Python + uv + Gemini API basics
↓
Week 2: Data pipelines + embeddings + ChromaDB (RAG foundation)
↓
Week 3: LangChain + LangGraph + orchestrator agent
↓
Week 4-5: Specialist agents + memory + prompt engineering + evaluation
↓
Week 6: Integration with backend (AIService bridge)
↓
Week 7-8: Evaluation metrics + documentation + portfolio polish

```

What You'll Build (Spec by Spec)

Spec AI-1: Data Ingestion Pipeline (Week 2)

What: Download the Bitext customer support dataset from HuggingFace, clean it, split it into categories, chunk it, generate embeddings, and store everything in ChromaDB.

Key concepts you'll learn:

- Loading datasets with HuggingFace `datasets` library
- Text chunking strategies (why 500 tokens? why overlap?)
- Embedding models (what is `all-MiniLM-L6-v2`? why does it run on CPU?)
- Vector databases (ChromaDB what it stores, how similarity search works)

Output files:

```

ai/
├── pipelines/
│   ├── data_loader.py      ← Download + clean + categorize dataset
│   ├── embeddings.py       ← Generate embeddings with sentence-transformers
│   └── knowledge_base.py   ← ChromaDB operations (store, retrieve, delete)
└── data/
    ├── raw/                ← Original dataset files
    ├── processed/          ← Cleaned, categorized JSON files
    └── README.md           ← Data documentation

```

Example code pattern:

```

# ai/pipelines/data_loader.py
from datasets import load_dataset

def load_and_categorize():
    """Load Bitext dataset and split into billing/technical/general."""
    dataset = load_dataset(
        "bitext/Bitext-customer-support-llm-chatbot-training-dataset"
    )

```

```

categories = {
    "billing": [],      # payment, refund, invoice, subscription
    "technical": [],    # bug, error, setup, feature
    "general": [],      # greeting, feedback, complaint
}

for row in dataset["train"]:
    intent = row["intent"]
    if intent in BILLING_INTENTS:
        categories["billing"].append(row)
    elif intent in TECHNICAL_INTENTS:
        categories["technical"].append(row)
    else:
        categories["general"].append(row)

return categories

```

Spec AI-2: RAG Retrieval Pipeline (Week 3)

What: Build the retrieval function that takes a user query, finds the most relevant knowledge base chunks, and returns them as context for the LLM.

Example code pattern:

```

# ai/pipelines/knowledge_base.py
import chromadb

def retrieve(query: str, category: str, top_k: int = 5) -> list[dict]:
    """Find the most relevant FAQ chunks for a user query."""
    client = chromadb.PersistentClient(path="./chroma_data")
    collection = client.get_collection(f"{category}_kb")

    results = collection.query(
        query_texts=[query],
        n_results=top_k,
    )

    return [
        {
            "title": meta["title"],
            "chunk": doc,
            "relevance_score": 1 - dist,
        }
        for doc, meta, dist in zip(
            results["documents"][0],
            results["metadatas"][0],
            results["distances"][0],
        )
    ]

```

Spec AI-3: Orchestrator Agent (Week 3)

What: Build the routing agent that classifies user intent and sends the query to the correct specialist.

Example code pattern:

```
# ai/agents/orchestrator.py
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate

ORCHESTRATOR_PROMPT = """You are a customer support routing agent.
Classify the customer's intent into exactly one category:
- BILLING: payment, refund, invoice, subscription, pricing
- TECHNICAL: bug, error, setup, feature, how-to
- GENERAL: greeting, feedback, complaint, general inquiry

Respond with ONLY the category name.

Examples:
User: "I was charged twice" → BILLING
User: "The app crashes on startup" → TECHNICAL
User: "Thanks for your help!" → GENERAL

User: "{message}"
Category: """"

async def classify_intent(message: str) -> str:
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash")
    prompt = ChatPromptTemplate.from_template(ORCHESTRATOR_PROMPT)
    chain = prompt | llm
    result = await chain.ainvoke({"message": message})
    return result.content.strip().upper()
```

Spec AI-4: Specialist Agents + Memory (Weeks 4–5)

What: Build 3 specialist agents, each with their own system prompt, knowledge base access, and conversation memory.

Spec EVAL-1: Evaluation Suite (Week 7)

What: Build automated quality measurement for the AI pipeline using RAGAS metrics.

Prompt Engineering Guide

Writing good prompts is 80% of the work in building LLM applications. Here are the patterns you'll use:

Pattern 1: System + Context + History + Query

[System Prompt]	← "You are a billing support agent. Be empathetic..."
[Retrieved Docs]	← "FAQ: Refund Policy says..."

```
[Conv. History]    <- "User: Hi | Agent: Hello! | User: I have a billing issue"
[Current Query]    <- "I was charged twice for my subscription"
```

Pattern 2: Few-Shot Examples

```
[System] Classify the intent.
[Example 1] "I was charged twice" → BILLING
[Example 2] "App crashes on upload" → TECHNICAL
[Example 3] "Your product is great" → GENERAL
[Query] "Can I get a refund?" → ???
```

Pattern 3: Chain-of-Thought

```
[System] Think step by step before answering.
[Query] "I can't access my account and I was charged"
[Expected] "Step 1: This involves account access (technical) AND billing.
            Step 2: The billing concern is more urgent.
            Step 3: Route to BILLING."
```

5. Role Guide: Software Engineer

Your Learning Journey

```
Week 1: TypeScript + React + Vite + FastAPI basics
  ↓
Week 2-3: FastAPI backend (routes, SQLAlchemy, WebSocket)
  ↓
Week 4-5: React chat UI (components, state, streaming, styling)
  ↓
Week 6: Integration with AI pipeline (AIService bridge)
  ↓
Week 7: Admin dashboard + bug fixes + polish
  ↓
Week 8: Docker + deployment (Render, Vercel, GitHub Actions)
```

What You'll Build (Spec by Spec)

Spec API-1: FastAPI Backend (Weeks 2–3)

What: Build the REST API that handles chat messages, manages conversations, connects to the AI pipeline, and stores everything in SQLite.

Output files:


```

apps/api/
├── src/
│   ├── main.py           ← FastAPI app entry point
│   ├── config.py         ← Pydantic Settings (env vars)
│   ├── database.py       ← SQLAlchemy engine + session
│   ├── models/
│   │   ├── conversation.py ← Conversation table
│   │   └── message.py     ← Message table
│   ├── routes/
│   │   ├── chat.py        ← POST /api/v1/chat
│   │   ├── conversations.py ← CRUD for conversations
│   │   └── health.py      ← GET /health
│   ├── services/
│   │   ├── chat_service.py ← Business logic for chat
│   │   └── ai_service.py   ← Bridge to AI pipeline
│   └── middleware/
│       └── error_handler.py ← Global error handling
├── tests/
├── pyproject.toml
└── .env.example

```

Example code patterns:

```

# apps/api/src/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="Customer Support AI", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

from src.routes import chat, conversations, health
app.include_router(health.router)
app.include_router(chat.router, prefix="/api/v1")
app.include_router(conversations.router, prefix="/api/v1")

```

```

# apps/api/src/models/message.py
from sqlalchemy import Column, String, Text, DateTime, ForeignKey, Enum
from src.database import Base
import uuid
from datetime import datetime

class Message(Base):

```

```
__tablename__ = "messages"

id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
conversation_id = Column(String, ForeignKey("conversations.id"))
role = Column(Enum("user", "assistant", "system", name="message_role"))
content = Column(Text, nullable=False)
agent_used = Column(String, nullable=True)
sources = Column(Text, nullable=True) # JSON string
created_at = Column(DateTime, default=datetime.utcnow)
```

Spec WEB-1: React Chat UI (Weeks 4–5)

What: Build a modern, dark-mode chat interface with conversation management, streaming responses, and source citations.

Output files:

```
apps/web/
├── src/
│   ├── components/
│   │   ├── ChatWindow.tsx      ← Main chat area
│   │   ├── MessageBubble.tsx  ← Individual message
│   │   ├── ChatInput.tsx      ← Input bar + send button
│   │   ├── Sidebar.tsx        ← Conversation list
│   │   ├── SourceCitation.tsx  ← Collapsible sources
│   │   ├── AgentBadge.tsx      ← "Routed to Billing"
│   │   ├── TypingIndicator.tsx ← Three dots animation
│   │   └── ThemeToggle.tsx     ← Dark/light switch
│   ├── services/
│   │   └── api.ts              ← API client functions
│   ├── hooks/
│   │   └── useChat.ts          ← Chat logic hook
│   ├── types/
│   │   └── index.ts            ← TypeScript interfaces
│   ├── App.tsx
│   ├── App.css
│   └── main.tsx
```

Spec DEPLOY-1: Production Deployment (Week 8)

What: Containerize the app with Docker and deploy to free hosting.

6. The Runbook Workflow (How We Work)

What Is a "Runbook"?

A runbook is a **documentation folder** within your project that acts as the "brain" of your development process. It contains:

- **Specs** detailed build plans for each feature (the "how")
- **ADRs** records of architectural decisions (the "why")
- **Delivery tracking** what's done, what's in progress, what's next
- **Session context** so your AI coding agent can resume where you left off

This workflow was adapted from the **SGA Deploy Runbook**, a professional production system used by an AI engineering team to build and ship a hotel AI assistant.

Why Use a Runbook? (The Agentic Advantage)

When you use an AI coding agent (like Antigravity), the agent has no memory between sessions. Without a runbook:

```
Session 1: AI agent builds feature A. Understands context perfectly.
Session 2: AI agent has ZERO memory. Asks "what are we building?" again.
Session 3: AI agent makes a decision that contradicts Session 1.
Result: Frankenstein codebase. Inconsistent decisions. Wasted time.
```

With a runbook:

```
Session 1: AI agent builds feature A. Updates SPEC-CONTEXT.md.
Session 2: AI agent reads SPEC-CONTEXT.md → knows exactly where we left off.
           Reads SPEC-INDEX.md → knows what's done and what's next.
           Reads ADRs → knows WHY decisions were made.
Result: Consistent, professional codebase. Zero context loss.
```

The Four Pillars

Pillar 1: Specs (How to Build)

A spec is a detailed plan for a single piece of work. Every spec has 7 sections:

SPEC

1. Problem Statement

← Why does this exist?

2. Current State

← What's here now?

3. Proposed Design

← What will we build?

4. Build Plan

← Step-by-step phases

5. Verification List

← How to confirm done

6. Out of Scope

← What we WON'T do

7. Cross-References

← Related specs/ADRs

Why 7 sections?

- **Sections 1–2** force you to understand the problem before jumping to solutions.
- **Section 3** is your design review the AI agent and your teammate review this before any code is written.
- **Section 4** breaks work into small, committable phases with time estimates.
- **Section 5** prevents "is it done yet?" ambiguity when all boxes are checked, it's done.
- **Section 6** prevents scope creep the #1 killer of side projects.
- **Section 7** prevents contradictions between specs.

Pillar 2: ADRs (Why We Decided)

An ADR (Architecture Decision Record) is a short document that records a significant "why" decision. ADRs are **immutable** once accepted, they are never rewritten, only superseded.

Example: ADR-0001 Use Gemini Free Tier

Why not OpenAI? → No permanent free tier. Trial credits expire.

Why not local Ollama only? → Too slow on CPU (40-100s per response).

Why not Claude? → No free API tier.

Decision: Gemini primary, Groq fallback, Ollama offline.

When to write an ADR:

- Choosing between two technologies (Gemini vs OpenAI)
- Making an architectural decision (SQLite vs Postgres)
- Deciding to defer something (fine-tuning → deferred, use RAG instead)
- Any decision your future self might ask "why did we do this?"

Pillar 3: Delivery Tracking (What's Done)

Three files work together to track progress:

DELIVERY-PLAN.md	→ What we're building (14 deliverables)
DELIVERY-STATUS.md	→ Live checklist (☒ done / ☐ not started)
SPEC-INDEX.md	→ Spec-level detail (status, prereqs, dependencies)

Pillar 4: Session Context (AI Agent Memory)

SPEC-CONTEXT.md is the **most important file** for agentic coding. It's a living document that tells your AI coding agent:

1. What the project is (60-second orientation)
2. What's been done (status snapshot with commit hashes)
3. What was learned (gotchas, bugs, decisions made during execution)
4. What decisions are locked (ADRs that constrain future work)
5. What to do next (the process)

You MUST update this file at the end of every coding session.

The Workflow Loop

ONE SPEC PER TURN

① ORIENT

Read SPEC-CONTEXT.md → SPEC-INDEX.md → identify next

② DISCUSS

Talk to your teammate about scope. Lock decisions.
If it's an AI decision, write an ADR.

③ DRAFT

Write the spec (7 sections). Save to docs/specs/.
Update SPEC-INDEX.md → status: ☐

④ REVIEW

Teammate reviews the spec. AI agent reviews the spec.
Approve or request changes.

⑤ EXECUTE

Write code per the Build Plan. One phase at a time.
Commit after each phase.

⑥ VERIFY

Run every item in the Verification Checklist.
All boxes checked? → spec is done.

⑦ UPDATE

Move spec to docs/specs/resolved/.
Update SPEC-INDEX.md → status: ☐
Update DELIVERY-STATUS.md → deliverable ☒
Update SPEC-CONTEXT.md → refresh status snapshot.
Write journal entry if end of week.

⑧ COMMIT

Meaningful commit message: "feat(AI-1): data pipeline"

→ Repeat for the next spec.

Daily Standup (Async)

Every day, each person posts a quick update (Discord, Slack, WhatsApp):

- Yesterday: Finished AI-1 Phase 2 (embedding pipeline). 1,500 chunks embedded.
- Today: Starting AI-1 Phase 3 (ChromaDB storage + retrieval testing).
- Blockers: None.

7. Tech Stack & Installation Guide

AI Engineer Stack

Tool	Install Command	Purpose
Python 3.11+	Via uv (auto-installs)	Core language
uv	<code>powershell -c "irm https://astral.sh/uv/install.ps1 \& iex"</code>	Fast package manager
Google Gemini API	Sign up at aistudio.google.com	Primary LLM
Groq API	Sign up at console.groq.com	Fallback LLM
Ollama	Download from ollama.com	Local/offline LLM
LangChain	<code>uv pip install langchain langchain-google-genai langchain-groq</code>	LLM framework
LangGraph	<code>uv pip install langgraph</code>	Agent orchestration
ChromaDB	<code>uv pip install chromadb</code>	Vector database
Sentence-Transformers	<code>uv pip install sentence-transformers</code>	Embedding model
HuggingFace Datasets	<code>uv pip install datasets</code>	Load datasets
RAGAS	<code>uv pip install ragas</code>	Evaluation metrics

Software Engineer Stack

Tool	Install Command	Purpose
Node.js 20+	Download from nodejs.org	JavaScript runtime
TypeScript	Included with Vite template	Type-safe JS
React + Vite	<code>npx create-vite@latest . --template react-ts</code>	Frontend framework
FastAPI	<code>uv pip install fastapi uvicorn</code>	Backend framework
SQLAlchemy	<code>uv pip install sqlalchemy</code>	Database ORM
Pydantic	Included with FastAPI	Data validation
python-dotenv	<code>uv pip install python-dotenv</code>	Environment vars
Docker	Download from docker.com	Containerization

8. Open-Source Datasets

Primary: Bitext Customer Support LLM Dataset

- **Source:** HuggingFace (bitext/Bitext-customer-support-llm-chatbot-training-dataset)
- **Size:** 26,872 question-answer pairs
- **Coverage:** 27 intents across 10 categories
- **License:** Apache 2.0 (free for any use)

How we categorize intents:

Agent	Intents
Billing	check_invoice, get_refund, payment_issue, place_order, cancel_order, track_order, track_refund
Technical	create_account, delete_account, edit_account, recover_password, registration_problems, delivery_period
General	complaint, contact_human_agent, review, and all others

Secondary: Syncora.ai Conversations Dataset

- **Source:** HuggingFace (strova-ai/customer_support_conversations_dataset)
- **Use:** Multi-turn conversation test cases for evaluation
- **License:** MIT

Loading the Dataset

```
from datasets import load_dataset

dataset = load_dataset(
    "bitext/Bitext-customer-support-llm-chatbot-training-dataset"
)

print(dataset["train"][0])
# {'instruction': 'I need to cancel my order',
#  'intent': 'cancel_order',
#  'category': 'ORDER',
#  'response': 'I understand you need to cancel...'}
```

9. Getting Started (Day 1)

Step-by-Step Setup

Both team members:

```
# 1. Install uv
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

# 2. Create GitHub repo "ai-customer-support" and clone it
git clone https://github.com/YOUR_USERNAME/ai-customer-support.git
cd ai-customer-support

# 3. Create folder structure
mkdir -p docs/specs/resolved docs/specs/archive
mkdir -p docs/adr docs/journals docs/integrations docs/evaluation
mkdir -p apps/api/src/models apps/api/src/routes apps/api/src/services
mkdir -p apps/api/tests apps/web/src
mkdir -p ai/agents ai/pipelines ai/prompts ai/evaluation ai/tests
mkdir -p data/raw data/processed
```

AI Engineer:

```
# Get API keys from aistudio.google.com and console.groq.com
# Install Ollama from ollama.com, then:
ollama pull phi4-mini

# Set up AI environment
cd ai
uv venv
uv pip install langchain langchain-google-genai langchain-groq
uv pip install langgraph chromadb sentence-transformers
uv pip install datasets pandas ragas pytest
```

Software Engineer:

```
# Set up backend
cd apps/api
uv venv
uv pip install fastapi uvicorn sqlalchemy pydantic python-dotenv pytest

# Set up frontend
cd ../web
npx create-vite@latest . --template react-ts
npm install
npm run dev
```

10. Weekly Milestones

Week	AI Engineer	Software Engineer	Sync Point
------	-------------	-------------------	------------

Week	AI Engineer	Software Engineer	Sync Point
1	Install tools, get API keys, write ADRs, test LLM calls, explore dataset	Set up repo, init FastAPI + React, write DB schema	Review ADRs, agree on API contract
2	AI-1: Data pipeline (download, clean, chunk, embed, ChromaDB)	API-1: FastAPI routes (chat endpoint, conversation CRUD)	
3	AI-2: RAG retrieval. AI-3: Orchestrator agent	API-1: WebSocket/SSE streaming. Finish backend	
4	AI-4: Specialist agents + conversation memory	WEB-1: Chat UI layout, message components	
5	AI-4: Prompt tuning + guardrails + LLM fallback	WEB-1: Streaming, dark mode, source citations	
6	INT-1: Connect AI pipeline to FastAPI via AIService	INT-1: Integrate AIService into routes, E2E testing	Integration week
7	EVAL-1: RAGAS metrics, benchmark accuracy	Bug fixes, admin dashboard, feedback feature	Review evaluation
8	Documentation, demo video, blog post	DEPLOY-1: Docker, Render, Vercel, CI	Deploy together

11. Runbook File Reference

File	Purpose	Who Updates It
docs/SYSTEM_ARCHITECTURE.md	System design source of truth	Both
docs/DELIVERY-PLAN.md	14 deliverables	Set once
docs/DELIVERY-STATUS.md	Live progress checklist	Both
docs/specs/TEMPLATE.md	How to write a spec	Reference only
docs/specs/SPEC-INDEX.md	Master spec tracker	Both
docs/specs/SPEC-CONTEXT.md	AI agent session resumption	Both (every session)
docs/adr/README.md	ADR index and format	Both
docs/adr/0001-*.md	Individual ADRs	Decision maker
docs/integrations/api-contract.md	API contract	Both (must agree)
docs/journals/week-NN/STATUS.md	Weekly journal	Both

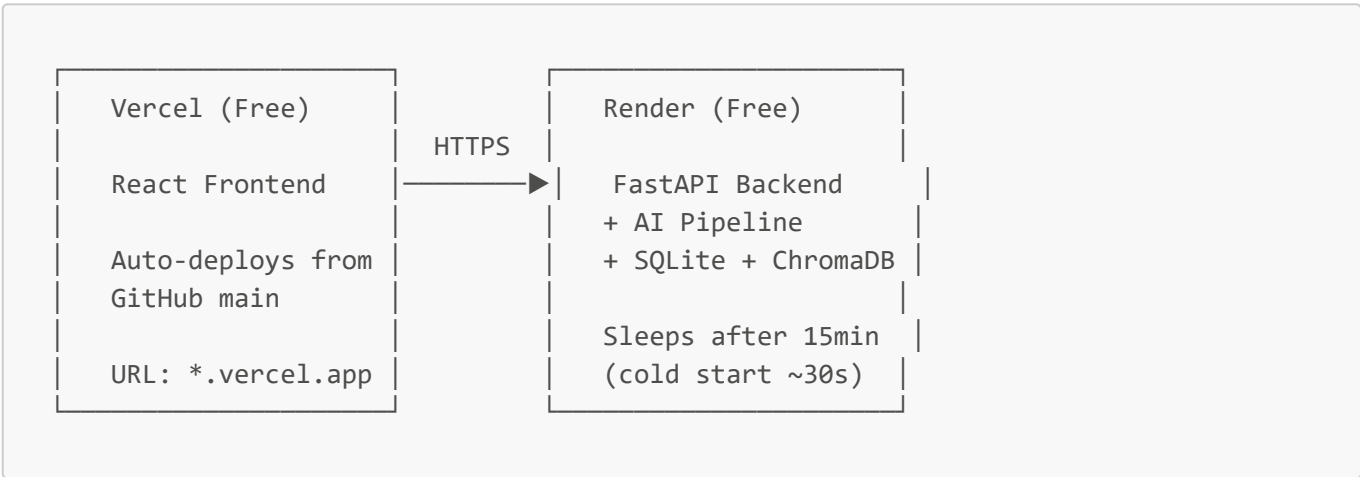
12. Evaluation & Quality Metrics

Metric	Target	How	Owner
--------	--------	-----	-------

Metric	Target	How	Owner
Intent classification accuracy	> 85%	Test orchestrator with 100 labeled queries	AI Eng
Response relevancy (RAGAS)	> 0.8	RAGAS eval on 50 test questions	AI Eng
Faithfulness (RAGAS)	> 0.8	Does the answer match retrieved sources?	AI Eng
Context precision (RAGAS)	> 0.75	Are retrieved docs relevant?	AI Eng
Average response latency	< 5 sec	End-to-end logging	Both
API endpoint reliability	> 99%	Automated tests passing	SWE

13. Deployment Guide

Production Architecture



Deploy Checklist

- ☐ Write `apps/api/Dockerfile` (multi-stage build with `uv`)
- ☐ Write `docker-compose.yml` for local testing
- ☐ Test: `docker-compose up --build` → everything works
- ☐ Sign up for Render → connect GitHub → deploy backend
- ☐ Sign up for Vercel → connect GitHub → deploy frontend
- ☐ Set environment variables on Render (`GEMINI_API_KEY`, etc.)
- ☐ Test live URL end-to-end

14. Portfolio Presentation Tips

GitHub README Structure

```
# 🤖 AI Customer Support System

> Multi-agent chatbot with RAG-powered knowledge base

[Live Demo](URL) • [Demo Video](URL) • [Blog Post](URL)
```

🌟 Features

🏗️ Architecture

🛠️ Tech Stack

📊 Evaluation Results

🚀 Getting Started

📖 What I Learned

Demo Video (2 minutes)

- 1. Opening the app (0:00 – 0:10)
- 2. Billing question → routed to Billing Agent (0:10 – 0:30)
- 3. Technical question → routed to Technical Agent (0:30 – 0:50)
- 4. Source citations expand (0:50 – 1:00)
- 5. Multi-turn follow-up (1:00 – 1:30)
- 6. Admin dashboard (1:30 – 1:50)
- 7. Architecture overview (1:50 – 2:00)

Interview Talking Points

- "Why RAG instead of fine-tuning?" → ADR-0003
- "How do you handle hallucinations?" → Guardrails + source citations
- "How do you evaluate the AI?" → RAGAS metrics + intent accuracy
- "What would you do differently at scale?" → Postgres, Redis, GPU inference
- "How did you collaborate?" → Spec-driven workflow, API contract, runbook

15. Glossary

Term	Definition
ADR	Architecture Decision Record documents why a technical decision was made
Agent	An AI component with a specific role, system prompt, and knowledge access
ChromaDB	An open-source vector database for storing and searching embeddings
Embedding	A vector (list of numbers) representing the meaning of text
FastAPI	A modern Python web framework for REST APIs
Groq	A cloud AI inference provider with a free tier
Intent	The purpose of a user's message (billing, technical, general)
LangChain	A Python framework for building LLM-powered applications

Term	Definition
LangGraph	A LangChain extension for multi-agent state machines
LLM	Large Language Model AI that generates text (e.g., Gemini, GPT)
Ollama	A tool for running LLMs locally on your computer
Orchestrator	The routing agent that classifies intent
RAG	Retrieval-Augmented Generation finding docs before generating
Runbook	A documentation system that acts as the project's "brain"
Spec	A detailed build plan for a single feature (7 sections)
SQLAlchemy	A Python library for database access using objects
SQLite	A lightweight, file-based database
uv	An ultra-fast Python package manager
Vector Database	A database for storing and searching embedding vectors
Vite	A fast JavaScript build tool for React

This guide was created on August 11, 2026. For the latest project state, check [docs/specs/SPEC-CONTEXT.md](#).